

CT70A2000

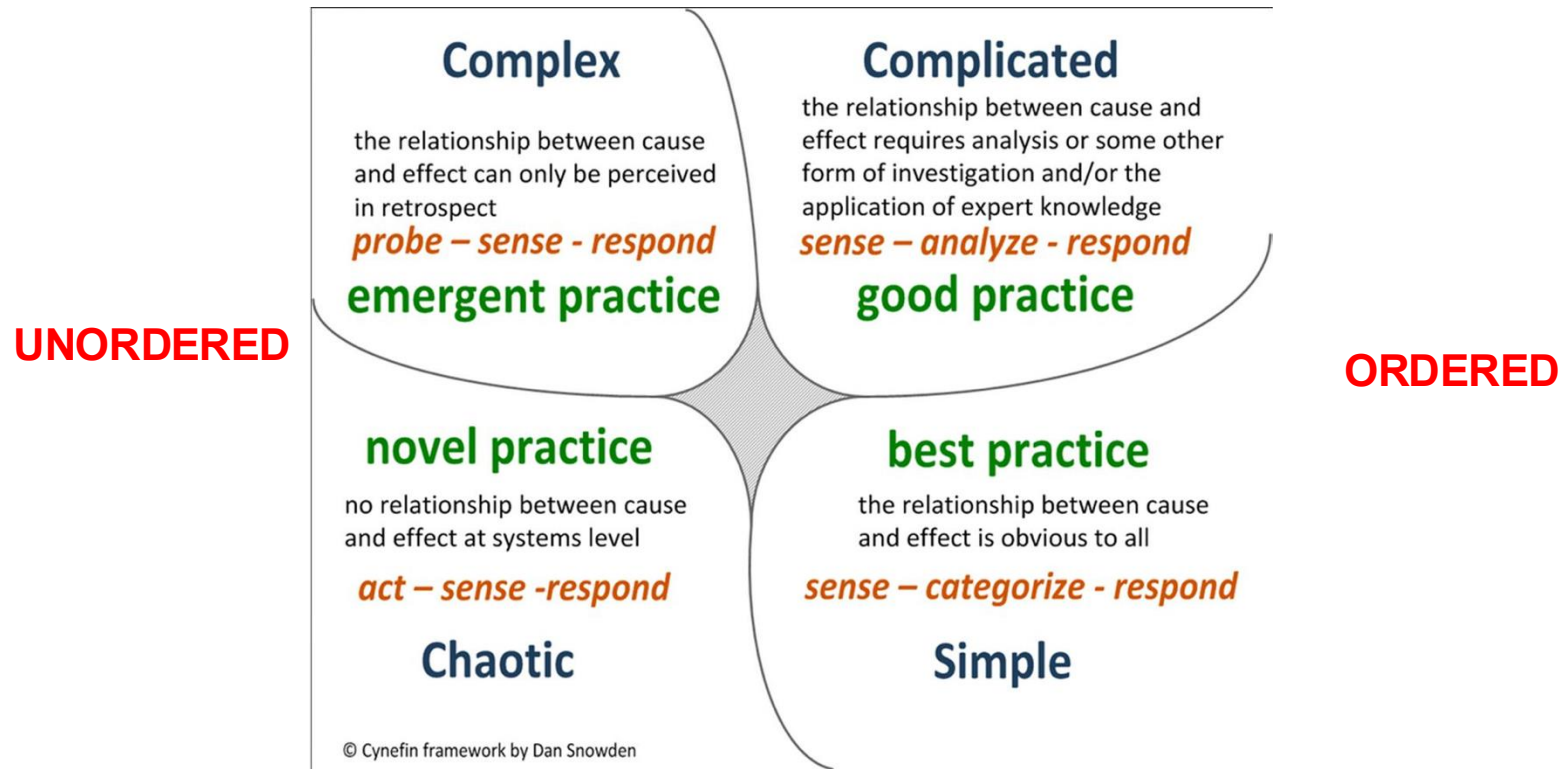
Requirements Engineering

Shola Oyedeji

Lecture 8:

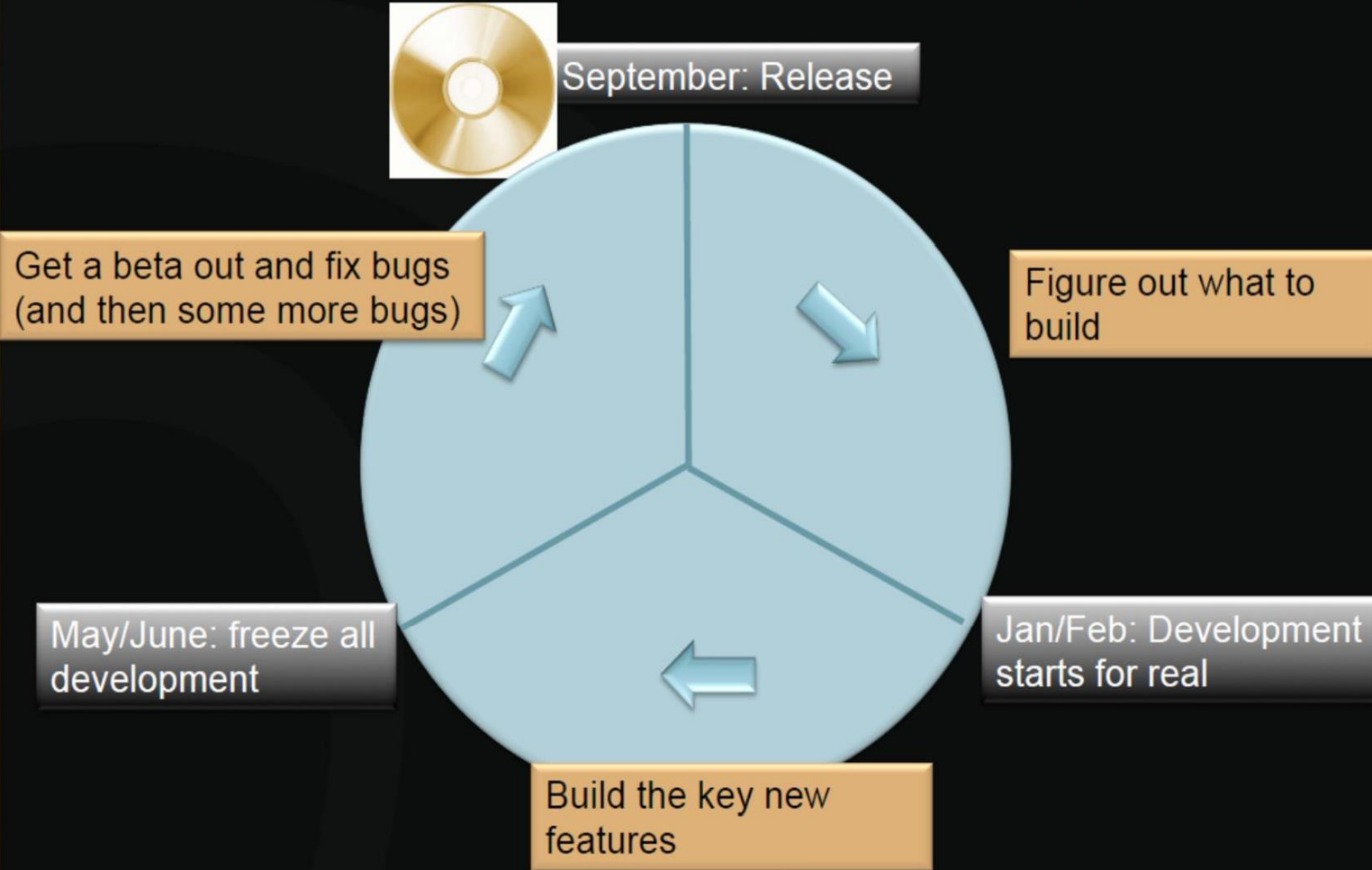
Towards Present and Future RE

CYNEFIN FRAMEWORK



D. J. Snowden and M. E. Boone, "A Leader's Framework for Decision Making", *Harvard Business Review*, vol. 85, 2007
<https://hbr.org/2007/11/a-leaders-framework-for-decision-making>

Old Development Process: Yearly Releases



Old Development Process: Problems

Lack of customer feedback



Heavy, top-down development process



Inefficient use of engineering resources



Low engagement of team members



**[The assumption that a]
reasonably well-defined set of
requirements exists, if only we
take the time to understand
them, is wrong**

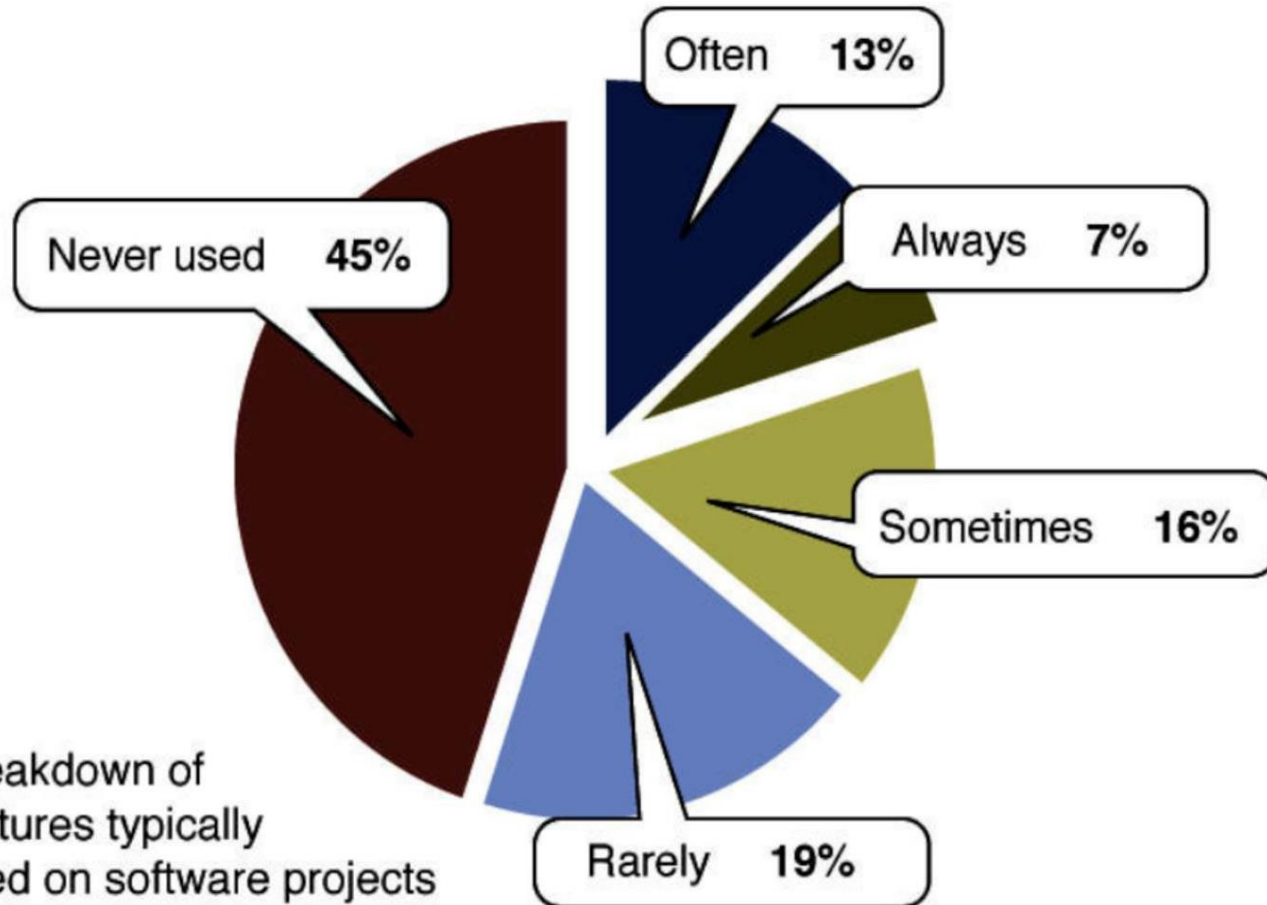
Dean Leffingwell

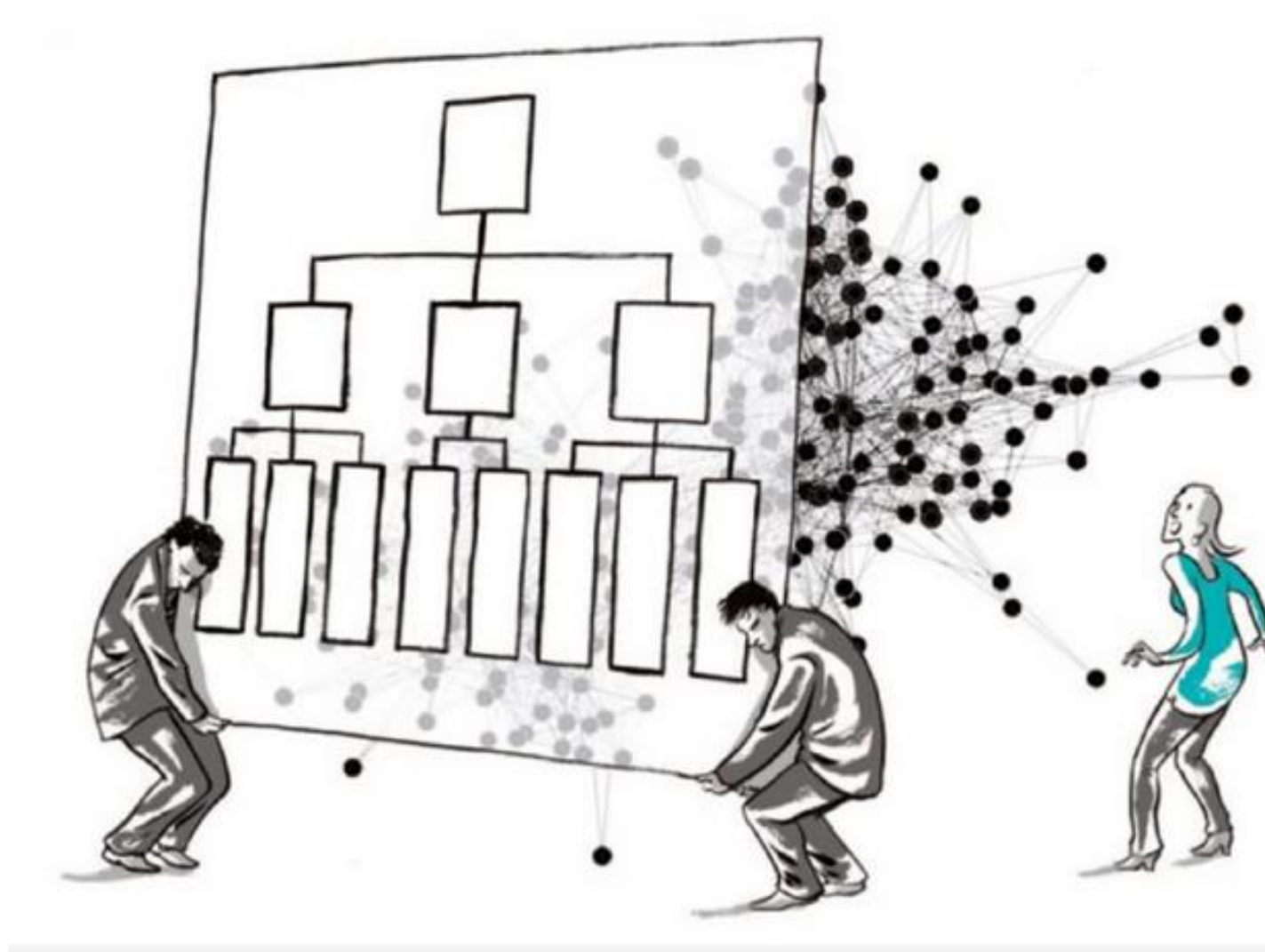
Source: Jan Bosch (2013): "Do As I Say; Not As I Do? From Requirements Engineering to Experimenting with Customers"
Keynote presentation in REFSQ 2013 conference

***IF WE DON'T MEET THESE REQUIREMENTS ...
WE ARE ALL GOING TO DIE!!!***



Featuritis

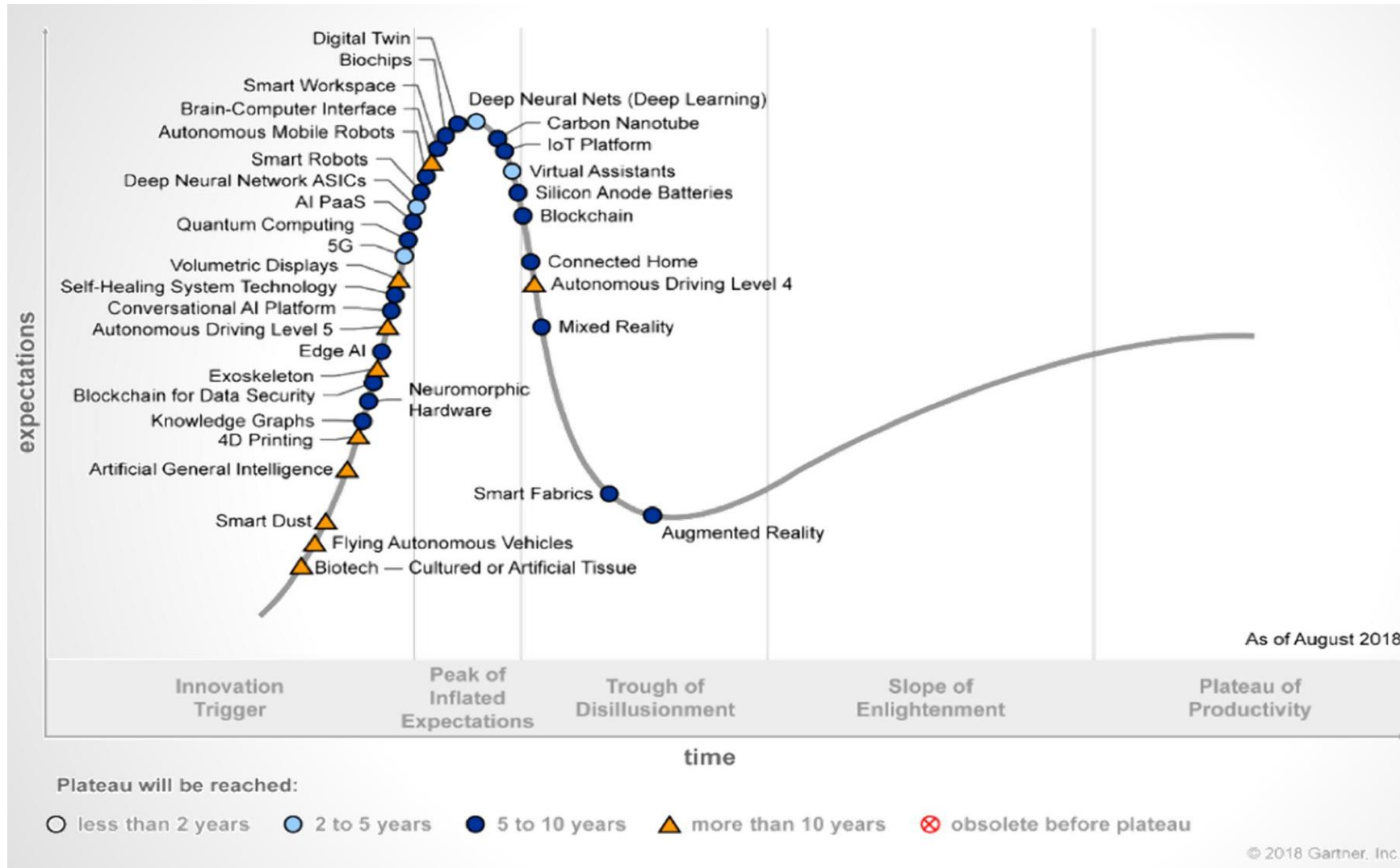




Source: Frederic Laloux's Reinventing Organizations, 2016

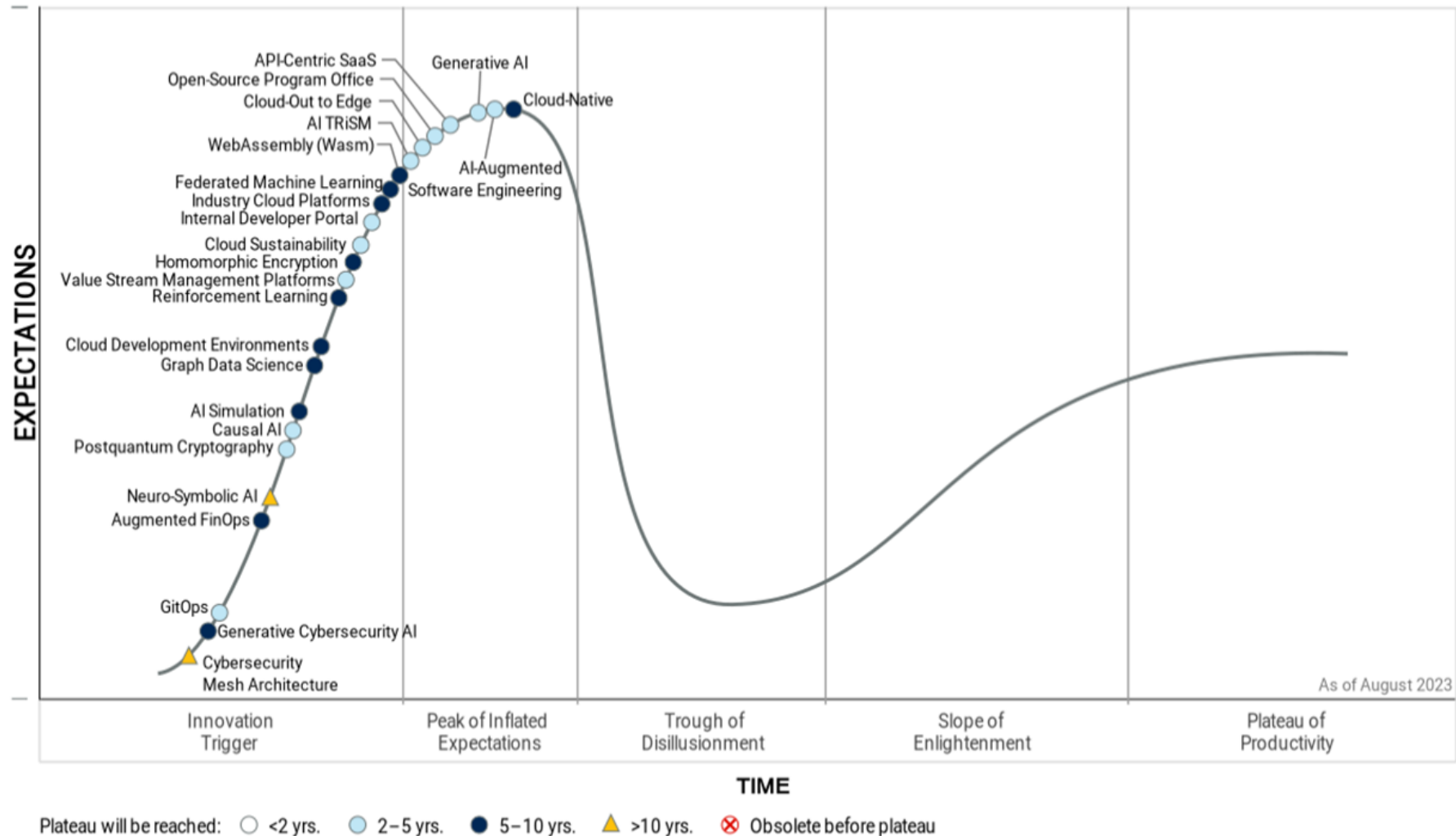
WORLD IS CHANGING...

(GARTNER HYPE CYCLE 2018)



WORLD IS CHANGING...

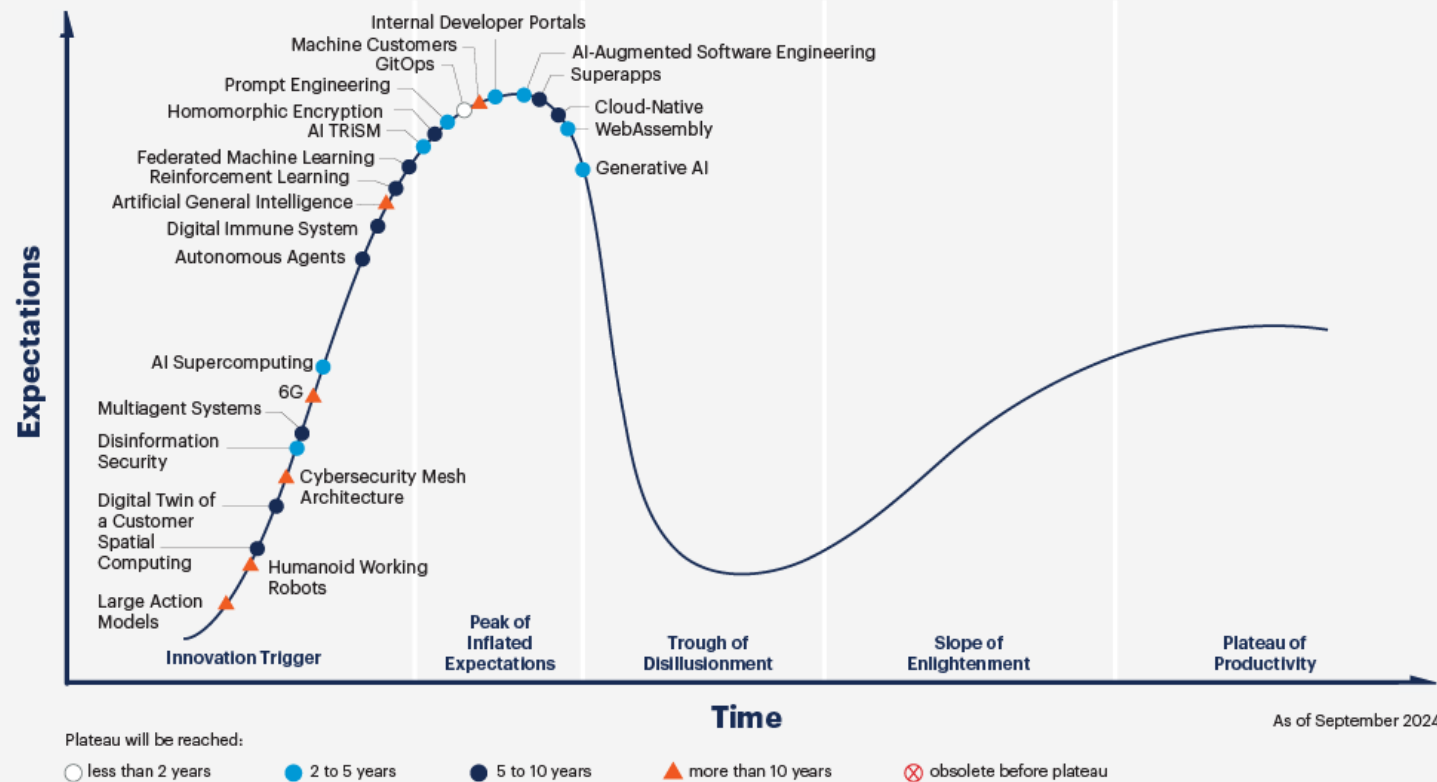
(GARTNER HYPE CYCLE 2023)



WORLD IS CHANGING...

(GARTNER HYPE CYCLE 2024)

Hype Cycle for Emerging Technologies, 2024



Source: Gartner

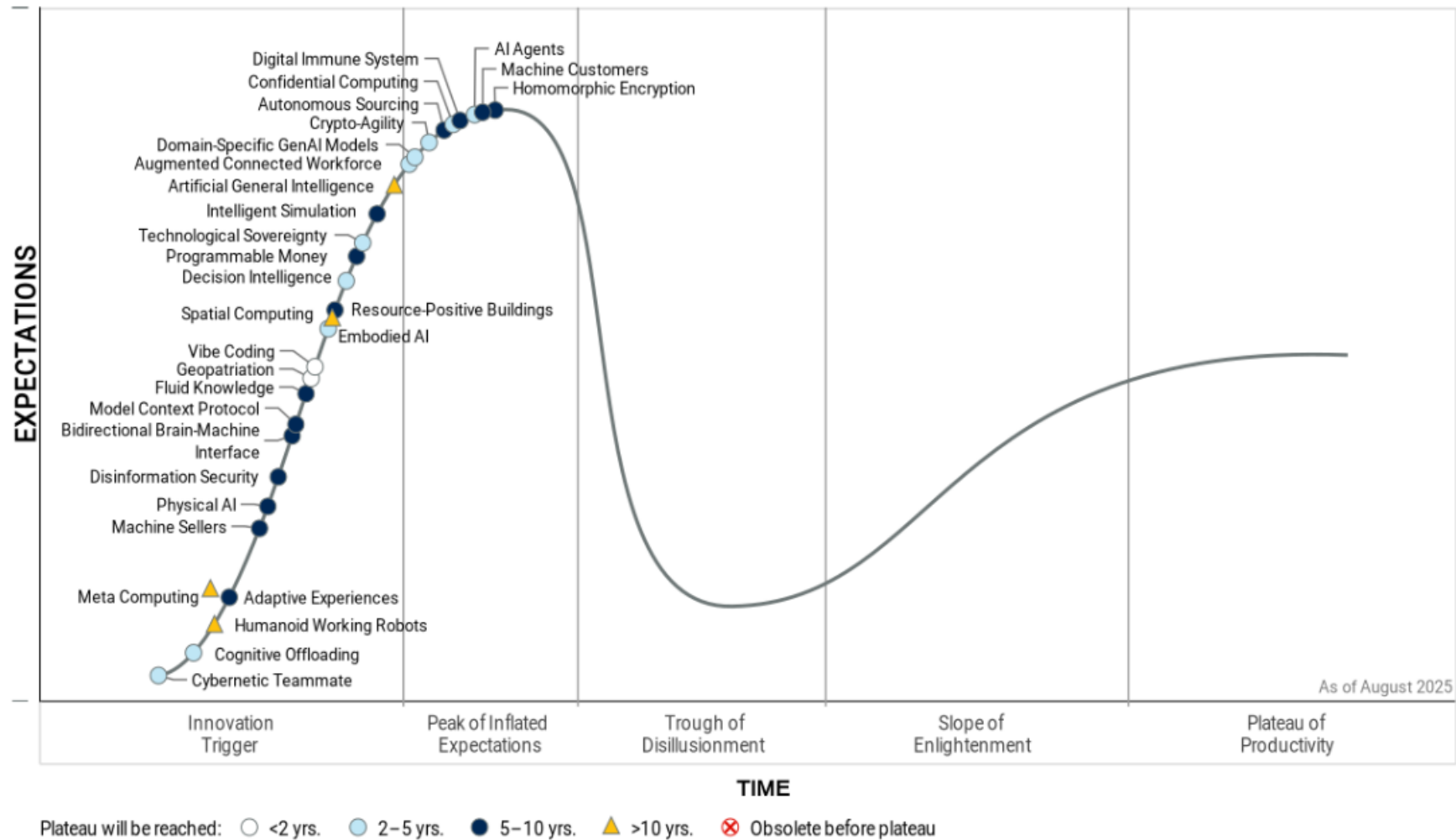
Commercial reuse requires approval from Gartner and must comply with the Gartner Content Compliance Policy on [gartner.com](https://www.gartner.com).

© 2024 Gartner, Inc. and/or its affiliates. All rights reserved. 3205434

Gartner

WORLD IS CHANGING...

(GARTNER HYPE CYCLE 2025)



SOFTWARE IS EVERYWHERE

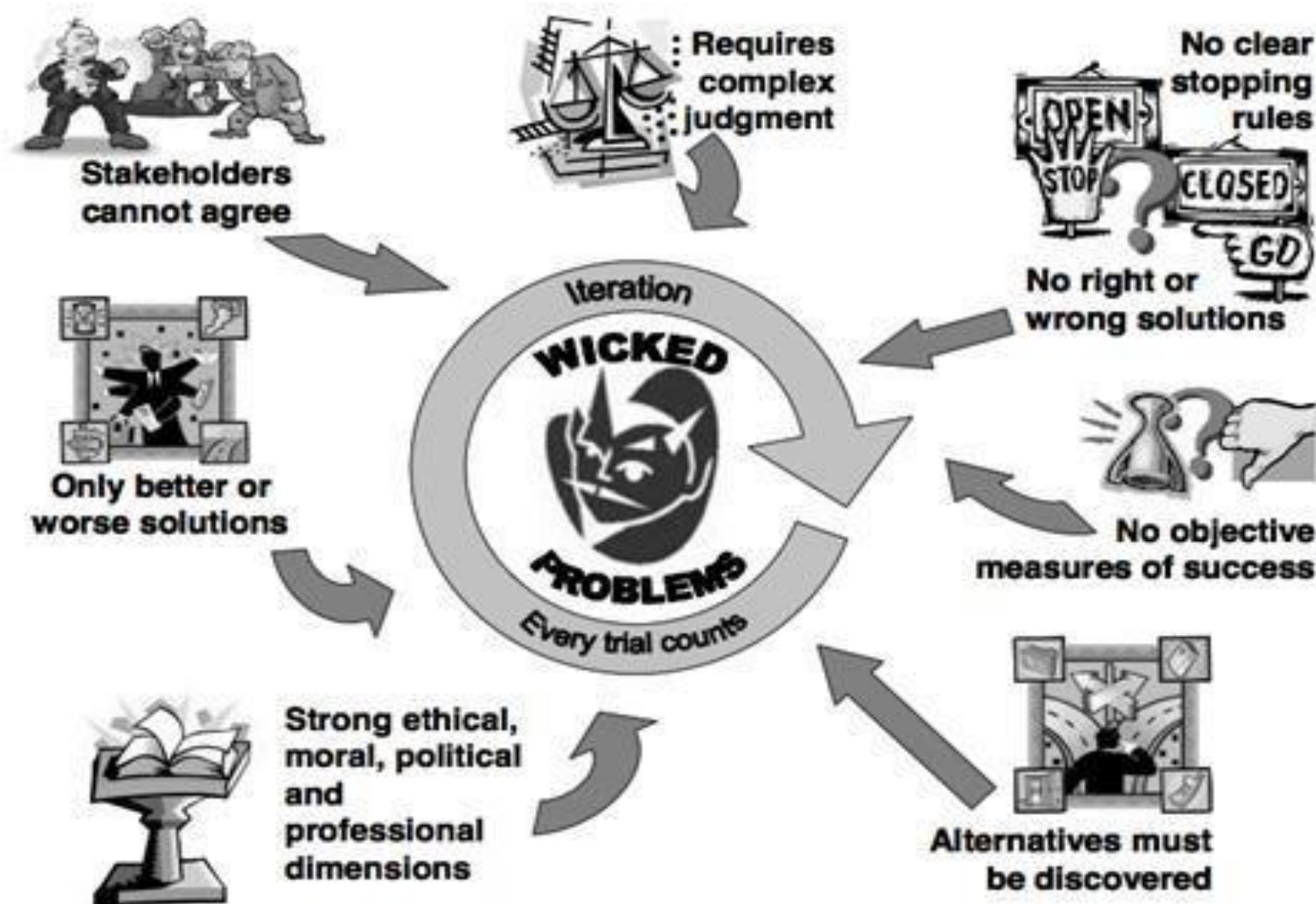


Trend: Products to Services

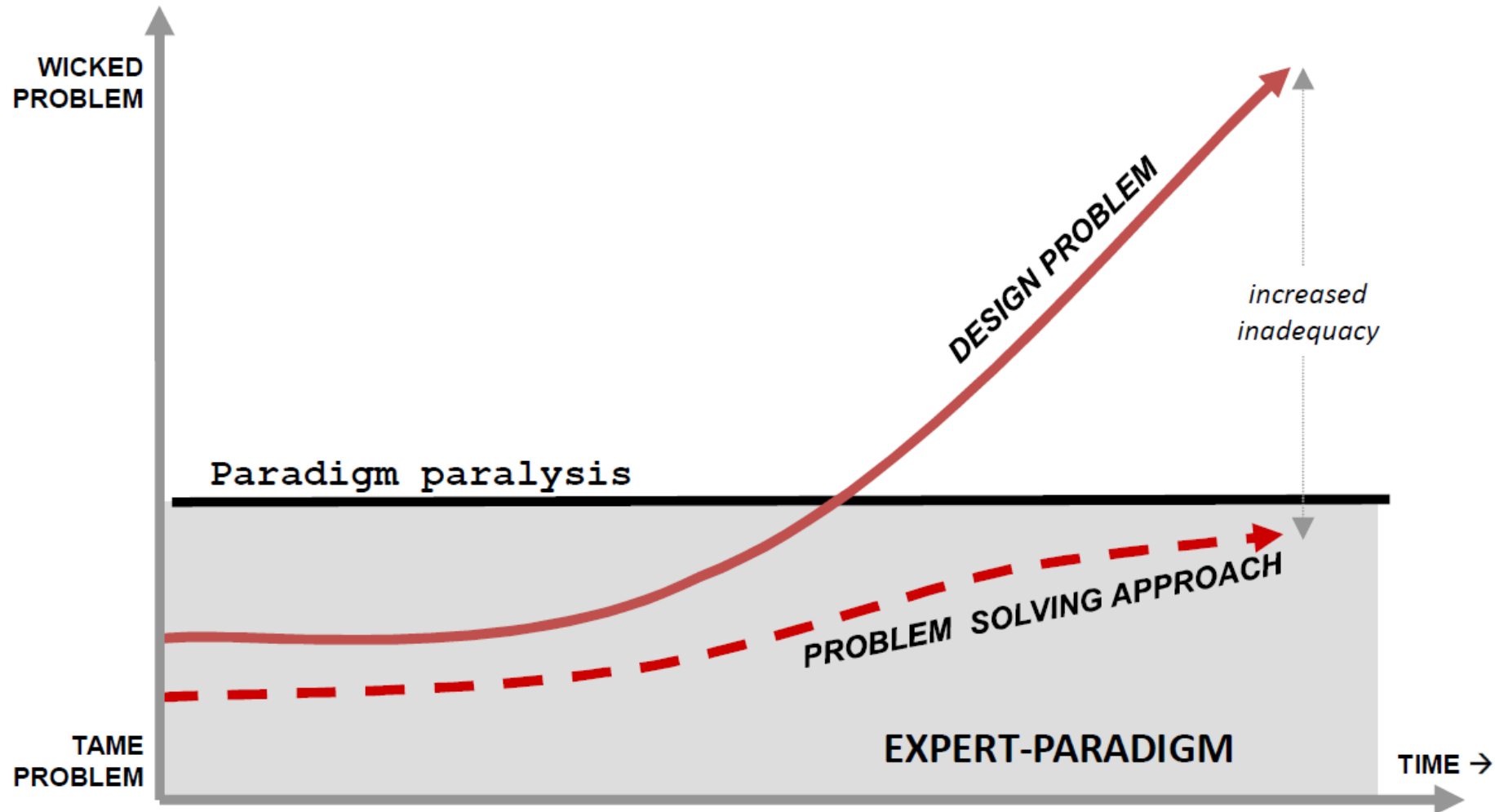


This requires continuous deployment throughout the lifetime of the product

WICKED PROBLEMS



WE NEED TO RECONSIDER OUR BELIEFS (BUT THIS IS HARD)



RE NEED TO EXPAND & EVOLVE

CONTEXTUAL INTELLIGENCE

- Ability to understand the nature of design problem
- Ability to understand the limits of traditional development methods
- Development of new methods to cope better with wicked problems



AGILE REQUIREMENT PROCESS

Agile Manifesto



Requirements Techniques for Agile Product Teams

1. Interviews
2. Focus Groups
3. Prototyping
4. Workshops
5. Questionnaires or Surveys
6. User Observation
7. Document Analysis
8. Interface Analysis
9. Brainstorming
10. Role-Play
11. Use Cases and Scenarios

TRADITIONAL VS. AGILE RE

Metric	Traditional development process	Agile development process	Study that reported
SDLC	Linear	Iterative	[9] [10]
Development style	Traditional methods are predictive (plan-driven)	Agile methods are adaptive	[11]-[14]
Documentation	Enough documentation to be able to answer all questions that might be asked in the future.	Light (replaced by face to face communication)	[7] [15]
Customer involvement	in traditional approaches the customer is mainly engaged during the early phase of the project	Agile methods engage the customer throughout the whole development process.	[13] [16]-[18]
Change	Resistance to change	Welcoming to change, even changes are brought in late in the project.	[19] [20]
Size	traditional methods are able to manage effectively large project	Manage effectively requirements in small projects but not in large ones.	[13]
Planning scale	Long term	Short term	[13]
Management	process oriented, command and control	People oriented, leadership and conformity.	[11] [12] [15] [21] [22]
Team organization	Pre- structured teams	Self organizing teams	[23]
Ownership	Ownership belongs to only project manager	Shared ownership	[19]
Prioritization	Requirements are typically prioritized once.	Prioritize feature lists repeatedly during development	[24]
Customer feedback	At the termination of the project	At the completion of every sprint	[25]
Risk identification	No risk identification.	Early identification and mitigation in every sprint.	[25]
Time between specification & implementation	Long	Short	[13]
Delivery	Delivering artifacts phase wise and delivery of working software at the end of project.	Demonstration and delivering working software et the end of every sprint.	[25]
Measure of Success	Conformance to plan	Business value delivered	[3]

ISSUES RESOLVED BY AGILE RE

Issues of RE in Waterfall	Resolved by RE in Agile
<i>Customer involvement:</i> Customers are involved only during the beginning of requirement gathering and analysis [40].	Customers are involved throughout the complete process.
<i>Prioritization of requirements:</i> Complete requirements for the full project are prioritized upfront and the prioritization is kept up through the project lifecycle, and reprioritization is arduous [47].	Priorities are setup for all iterations that offer opportunities for getting desirable results and customer satisfaction [48] [49].
<i>Documentation:</i> Totally emphasizes at properly gathering organizing and documenting all requirements and excludes any live meetings/conferences [14].	User stories are concise and provide to-the-point explanation of user demands, obviate the need for maintaining long SRS documents.
<i>Requirements validation:</i> Validation happens late in the life cycle.	Prototyping helps in providing the customer with a blueprint of the product, and therefore helps in validating the requirements [50].
<i>Communication:</i> It is a major factor in the delay and failure of software projects [40].	It provides regular interaction with customer and among teams.
<i>Over-scoping of requirements:</i> It is the cause of rework, which in turn causes further investment.	Developers receive a list of features that are constantly prioritized so the chance of having to repeat allocation in projects is minimized.
<i>Shall Argument:</i> The worst thing of waterfall RE is “shall” argument <i>i.e.</i> system shall do it, etc. [46].	Agile introduces the real time system.

ISSUES OF AGILE RE

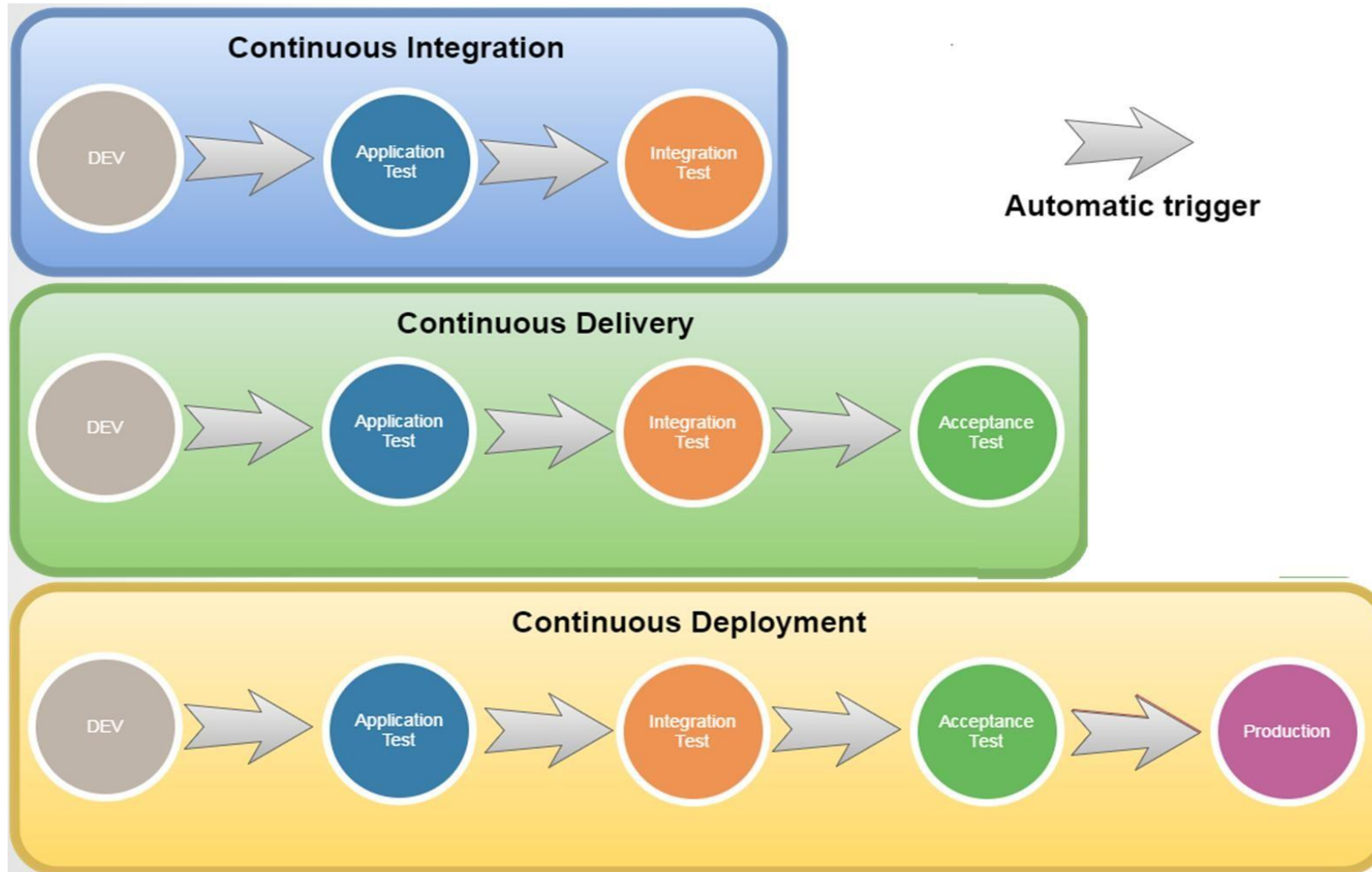
Table 8

Summary of challenges of agile RE.

Challenge	Description	Impact	Solutions
Minimal documentation (Cao & Ramesh, 2008)	User stories and product backlogs are the only documents in agile methods (Zhu, 2009)	Traceability issues (Zhu, 2009)	
Customer availability (Ramesh et al., 2010)	Availability of customer for requirements negotiation, clarification and feedback	Increase in rework	Surrogate customers (Ramesh et al., 2010)
Inappropriate architecture (Ramesh et al., 2010)	Inadequate infrastructure can cause problems during later project stages	Increase in cost	Code refactoring (Berry, 2002)
Budget and time estimation (Cao & Ramesh, 2008)	Initial estimates of time and cost are changed substantially by a change in requirements in subsequent stages	Project delays	Frequent communication
Neglecting non-functional requirements (NRFs)	User stories only satisfy system/product features	Over-budgeting System security, usability, performance at stake	Accurate modelling of user story NRF modelling approach (Farid & Mitropoulos, 2012b). The NORMATIC tool (Farid & Mitropoulos, 2012a)
Customer inability and agreement (Daneva et al., 2013; Ramesh et al., 2010)	Incomplete domain knowledge and in consensus among customer groups	Increase in rework	Creation of delivery stories to accompany user stories (Daneva et al., 2013)
		Increase in cost	Frequent communication Iterative RE (Ramesh et al., 2010)
Contractual limitations (Cao & Ramesh, 2008)	Fixed-price contracts do not allow changes	Increase in cost	
Requirements change and its evaluation	To find the consequences of requirements change	Increase in work delay	RE-KOMBINE framework (Ernst et al., 2013)

AGILE IS NOT
THE END OF
EVOLUTION

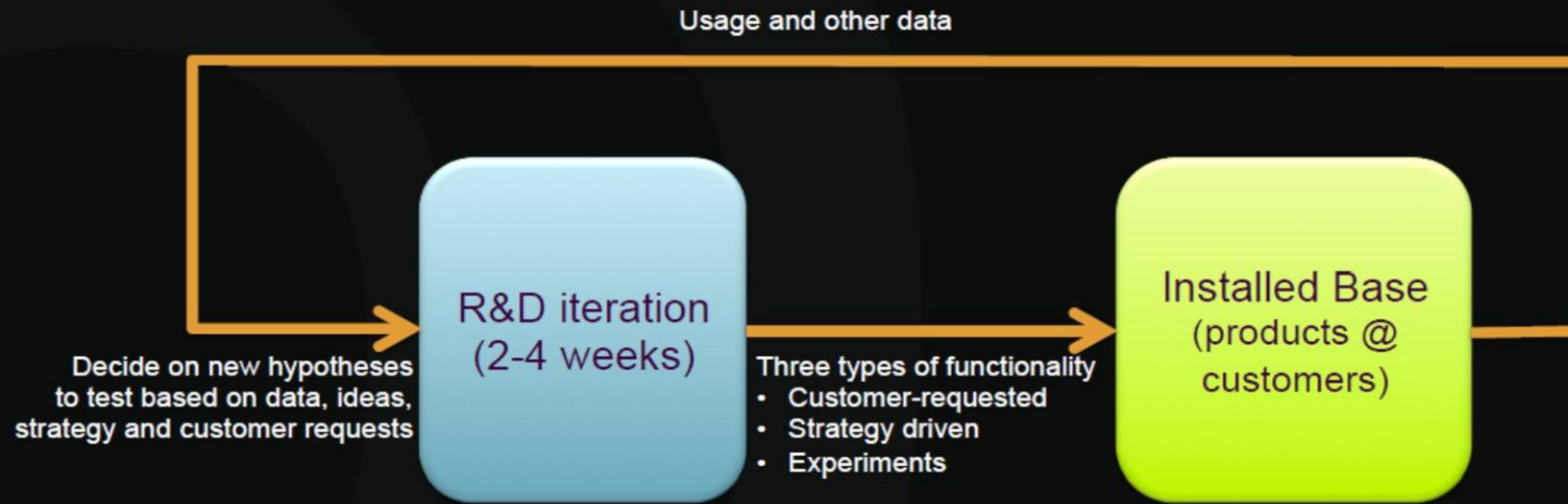
TREND: TOWARDS CONTINUOUS DEPLOYMENT



R&D as an Experiment System

Learning: the company running the most experiments against the lowest cost per experiment wins

Goal: increase the number of experiments (with customers) with an order of magnitude to ultimately accelerate organic growth



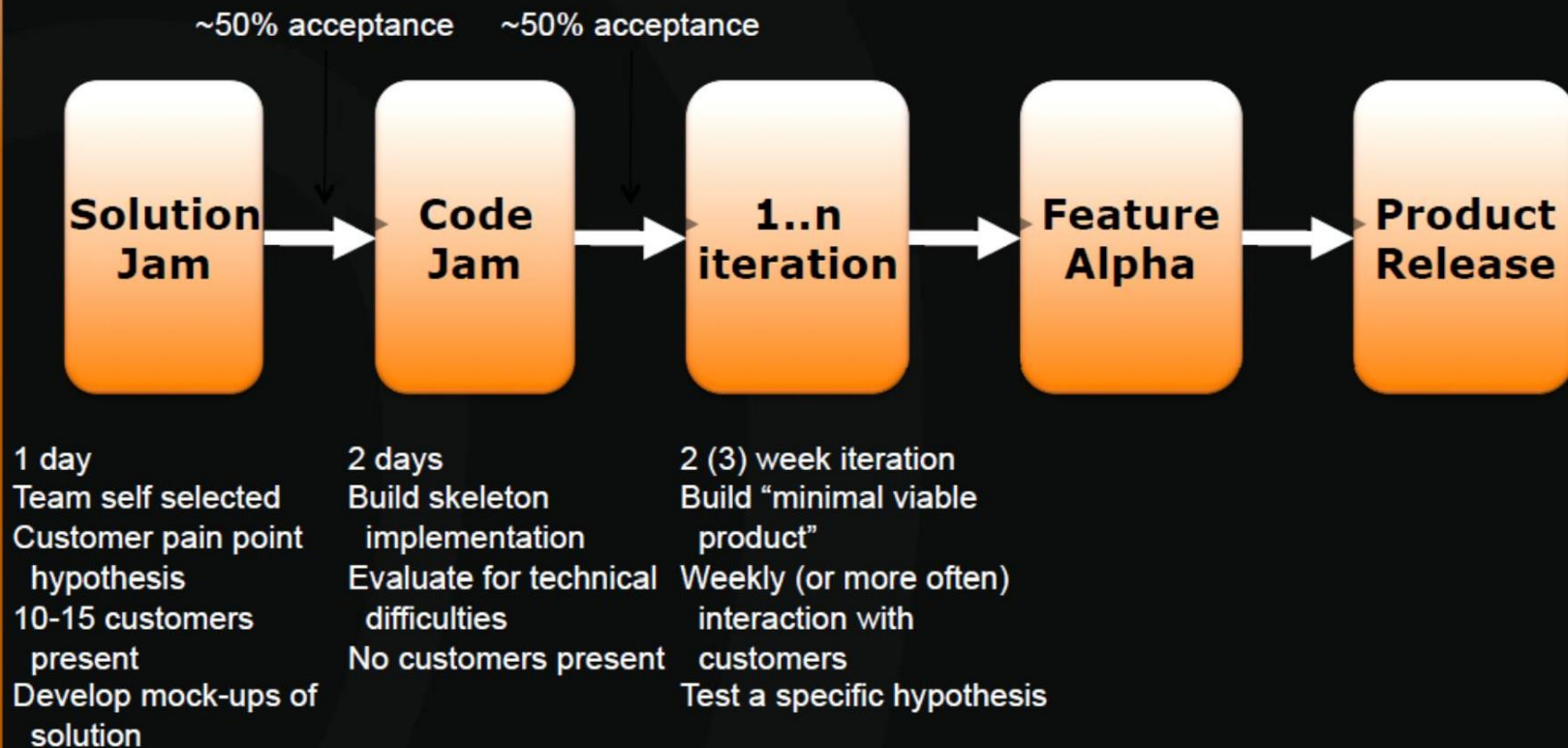
Decisions should be based on DATA, not opinions

Key Characteristics

- Teams are self-selected, self-directed and self-managed
- Teams are small (typically 3 people, but 1-6 is the range)
- Data instead of opinions: Customers are deeply involved in the process
- Three stages:
 - Concept testing: immediate customer feedback
 - Prototype testing: sandboxed release of system that does not disrupt the customer's data
 - Feature alpha: old release with new feature included at customer site

New Process: Overview

Top Down: Strategic Areas to Improve



The HYPEX model

